Phase 9: Testing (Vitest, React Testing Library, Playwright)

What we built

77 automated tests in three layers, all passing:

Layer	Tool	Files	Tests	Speed
Unit	Vitest	`portfolio`, `validation`, `blackScholes`, `optionChain`, `LocalOrderBook.test.ts`	57	~1 ms each
Integration	Vitest	`tradingStore.test.ts` (store + validation + P&L + matcher together)	7	~1 ms each
Component	Vitest + React Testing Library	`OrderForm.test.tsx`	7	~50 ms each
End-to-end	Playwright (real Chromium)	`e2e/order-flow.spec.ts`, 3 journeys × desktop + phone	6	~1 s each

Coverage of the tested logic: ~99% of lines. The first coverage run showed `optionChain.ts` at **0%**, so we added tests for it.

Commands (in `app/`)

Command	What it does
`npm test`	Unit + integration + component tests, once (~2 s)
`npm run test:watch`	The same, re-running **on every save** (keep it open while coding)
`npm run test:coverage`	Plus a coverage table (and `coverage/index.html`)
`npm run test:e2e`	Starts the app on port 5199 and runs Playwright on desktop + phone
`npx playwright test --ui`	Playwright's UI: **watch each step** in the browser, time-travel through it

Files

File	Role
`vite.config.ts` → `test`	Vitest settings: jsdom environment, setup file, which files are tests, coverage
`src/test/setup.ts`	Runs before every test file: adds DOM matchers, cleans up the DOM and localStorage
`src/**/*.test.ts(x)`	Tests live **next to** the code they test
`playwright.config.ts`	Starts the dev server, defines the desktop + Pixel 7 projects, keeps traces of failures
`e2e/fakeBinance.ts`	A **fake Binance**: intercepts the WebSocket and REST calls, and lets the test set prices
`e2e/order-flow.spec.ts`	Buy at market → P&L follows the price · limit order rests then fills · account survives reload

Concepts (theory only)

1. Why automated tests

- **Confidence to change code.** After a refactor, one command tells you nothing broke. Without tests, every change means re-clicking the whole app by hand.
- **Living documentation.** `it('fills at ITS price when the market reaches it')` says what the code is supposed to do.
- **Bugs found earlier are cheaper.** A failing test on your laptop costs seconds. A wrong P&L in production costs money and trust.
- **They make CI possible (Phase 10).** A robot can run tests on every push. It can't click around.

2. The testing pyramid

```
          /  E2E  \          few  · slow · realistic · brittle      (Playwright)
         /---------\
        / Component / \      some · medium                         (React Testing
Library)
       /  Integration  \
      /-----------------\
     /       Unit        \   many · fast · precise · cheap         (Vitest)
```

- **Unit**: one function or class, isolated. When one fails, you know **exactly** where the bug is.
- **Integration**: several real pieces together (our store + validation + P&L).
- **Component**: one UI component rendered and used like a user would.
- **End-to-end (E2E)**: the whole app in a real browser, along a user journey.

Why a pyramid: going up, tests get more realistic but **slower, flakier, and vaguer** when they fail ("the buy flow broke" vs "applyFill's average is wrong"). So you write many cheap tests at the bottom and a few important journeys at the top.

The "testing trophy" (Kent C. Dodds) is a popular variation for frontends: put the most weight on **integration/component** tests, since they give the most confidence per test.

3. Anatomy of a test (Vitest ≈ JUnit)

```
describe('applyFill: selling', () => {                // ≈ a JUnit test class
  it('realizes profit on the part sold', () => {      // ≈ a @Test method
    const p = applyFill(position(2, 150), 'BTCUSDT', 'SELL', 1, 180)   // Arrange + Act
    expect(p.realizedPnl).toBe(30)                                      // Assert
  })
})
```

- **AAA**: Arrange → Act → Assert.
- **One behaviour per test**, named as a sentence.
- **Matchers**:
  - `toBe` compares references or primitives.
  - `toEqual` does a deep compare.
  - `toMatchObject` checks a subset of fields.
  - `toBeCloseTo` is for floats (never `toBe` with floats).
  - `toHaveLength`, `toThrow`.
- `it.each` is a parameterized test, one table row per case (JUnit's `@ParameterizedTest`).
- `beforeEach` gives each test a fresh state. Tests must be **independent**: any order, any subset, same result.

Vitest vs Jest: the same API. Vitest reuses Vite's config and transforms, so TypeScript, JSX and imports just work, and it's much faster to start.

4. What makes a good test

- **Test behaviour, not implementation.** Assert on outputs and on what the user sees, not on private variables or how many times an internal function was called. Then you can refactor freely, and tests only fail when behaviour breaks.
- **Boundaries and edges**: exactly the minimum (passes), just below it (fails), zero, empty, float traps (`0.3 / 0.1`), expiry `T = 0`.
- **Deterministic**: no real clock, no randomness, no real network. Pass the time in as a parameter (`upcomingExpiries(now)`) or mock it.
- **Test data that can expose bugs.** We deliberately broke the P&L formula (dropped `× quantity`), and only **one** of the sell tests caught it, because the other used quantity **1**, where `x × 1 = x` hides the bug. Prefer values like 2 or 0.3.

- **Watch it fail.** A test that never failed might not test anything. Break the code on purpose once (we did) and check that the test goes red.

5. Testing maths you can't eyeball (Black–Scholes)

1. **Known answers**: textbook values (10.4506 / 5.5735).
2. **Invariants** (properties that must always hold): put–call parity, `N(x) + N(−x) = 1`, `putDelta = callDelta − 1`, 0 < call delta < 1.
3. **Limits and edges**: deep ITM delta → 1, deep OTM → 0, at expiry worth exactly the intrinsic value, with no `NaN`.

(Generating hundreds of random inputs and checking the invariants is called **property-based testing**, e.g. with fast-check.)

6. React Testing Library: test like a user

> "The more your tests resemble the way your software is used, the more confidence they can give you."

- **Render** the real component into **jsdom**, a fake browser DOM implemented in JavaScript, running inside Node. It's fast, but it has no real layout or painting.
- **Find elements the way a user or screen reader does**, in this priority: `getByRole('button', { name: 'Buy BTC' })` → `getByLabelText` → `getByPlaceholderText` → `getByText` → (last resort) `getByTestId`. Never CSS classes or component internals. Bonus: if a test can't find your button by role and name, a screen-reader user can't either. **Free accessibility checking.**
- **Query variants**:
  - `getBy…` throws if the element is missing. Use it when the element must exist.
  - `queryBy…` returns `null`. Use it to assert something is **absent**.
  - `findBy…` waits (async). Use it for things that appear later.
- **`userEvent`** simulates real typing (key by key: focus, keydown, input, keyup) and clicks. That's why our test could check that typing `1a.b5` leaves `1.5`. (`fireEvent` only dispatches a single event, so it's less realistic.)
- **jest-dom matchers** read like the user's view: `toBeInTheDocument()`, `toHaveValue('0.10000')`, `toHaveTextContent(...)`, `toBeDisabled()`.

7. Mocking: what to fake, and where

Mock = replace a real dependency with a controlled fake. - **Why:** speed, **determinism** (live prices change every second), and simulating situations that are hard to cause (a gap in the order book, a price crossing your limit). - **Where:** at the **boundary** of your system (network, clock, randomness, storage), not inside your own logic. Our E2E test fakes **Binance** with `page.routeWebSocket` and `page.route`. Every line of *our* code runs for real, and the app can't tell the difference. - **The danger:** a mock can drift from reality (Binance changes its format, and the mocks still pass). Keep fakes small and based on real samples, and keep a few checks against the real thing (we ran the real app in earlier phases). -

Vocabulary: - **stub**: returns canned answers - **mock**: also verifies how it was called - **spy**: wraps the real thing and records calls - **fake**: a working lightweight implementation, like our fake Binance - **Injecting control into stores:** in unit and integration tests we skip the network entirely and write prices straight into the store (`applyTickers`), the same entry point the real feed uses.

8. End-to-end with Playwright

- Drives **real browsers** (Chromium, Firefox, WebKit/Safari) through a single API.
- `webServer` in the config starts the app before the tests.
- **Projects** run the same tests in different setups. We use **desktop** and **Pixel 7**, so Phase 8's mobile tab layout is tested too.
- **Auto-waiting**: `expect(locator).toHaveText(...)` **retries until it's true** (up to 5 s). No `sleep(2000)` : fixed sleeps are either too short (flaky) or too long (slow).
- **Strict locators**: a locator matching 2 elements is an **error**. We hit this: `getByText('Live')` also matched "simulated against **live** prices". The fix was `{ exact: true }`. Being strict prevents tests from clicking the wrong thing.
- **Isolation**: each test gets a fresh **browser context** (empty cookies and localStorage), like a new incognito window.
- **Debugging**: `--ui` mode, `--headed` , and **traces** (a recording with DOM snapshots, network and console for each step) kept for failed tests.

9. Flaky tests

A **flaky** test sometimes passes and sometimes fails with no code change. It's the fastest way to make a team ignore its tests. Common causes and fixes: - Real network or real time → mock them. - Fixed `sleep` s → auto-waiting assertions. - Shared state between tests → fresh state per test. - Animations and order of async events → wait for the final state, not for steps.

We ran the E2E suite **3× in a row** to check it's stable. **Retries** (we allow 2, in CI only) are a safety net, not a fix.

10. Coverage

Coverage is the % of lines, branches and functions that the tests **executed**. - **Useful for finding untested code.** It showed `optionChain.ts` at 0%, so we wrote tests for it. - **Not proof of quality.** 100% coverage with weak assertions still misses bugs. Coverage says the line **ran**, not that the result was **checked**. Our broken-formula experiment shows this: the code was "covered" by several tests, but only one actually caught the bug. - **Branch coverage** (did both sides of every `if` run?) is stricter than line coverage. - Teams often set a minimum (e.g. 80%) on core logic, rather than chasing 100% everywhere.

11. TDD (test-driven development), briefly

Red → Green → Refactor: 1. Write a failing test for the next small behaviour. 2. Write the minimum code to pass it. 3. Clean up with the test as a safety net.

It works especially well for pure logic like validation and P&L. We wrote tests *after* the code (common when adding tests to an existing app), but used the same "make it fail first" check.

Try it yourself

1. `cd app && npm run test:watch`. Then in `portfolio.ts` change `(price - p.avgPrice) * quantity` to `(price - p.avgPrice)` and save: watch the test go red, and read the diff it prints. Undo it.
2. `npm run test:coverage`, then open `app/coverage/index.html` in a browser. Red lines were never run by any test.
3. `npx playwright test --ui`: click a test, press play, and scrub through the timeline to see the page at each step (desktop and Pixel 7).
4. In `order-flow.spec.ts`, change `'+50.00'` to `'+51.00'` and run `npm run test:e2e`. Read the failure: expected vs received, the call log, and the trace path. Then `npx playwright show-trace <path>`. Undo it.
5. In `OrderForm.test.tsx`, try `screen.getByText('Buy')`: it fails with "multiple elements found". Why is `getByRole('button', { name: 'Buy BTC' })` better?

Resume mapping

> "**Unit tests** (Vitest) for P&L and order validation" → `portfolio.test.ts`, `validation.test.ts`, plus Black–Scholes, the order book and the option chain "**React Testing Library** component tests" → `OrderForm.test.tsx` "**Playwright** end-to-end tests" → `e2e/order-flow.spec.ts` with a mocked Binance WebSocket, on desktop and mobile viewports Numbers you can quote: **77 tests**, **~99% line coverage** of trading logic, unit suite in **~2 s**, E2E in **~5 s**.

Interview Qs

1. **Explain the testing pyramid.** Many fast unit tests, fewer integration/component tests, few slow E2E tests. Higher levels are more realistic but slower, flakier and less precise about *where* a bug is.
2. **Unit vs integration vs E2E with examples from your project?** Unit: `applyFill` averaging. Integration: `placeOrder` + validation + matcher in the store. E2E: a real browser buys BTC and sees the P&L move.
3. **What does "test behaviour, not implementation" mean?** Assert on outputs and on what the user sees, not on internal state or calls. Tests then survive refactors and only fail on real regressions.
4. **Why does RTL prefer `getByRole`?** It finds elements the way users and assistive tech do, so tests don't break on styling or markup changes, and it surfaces accessibility problems.
5. **`getBy` vs `queryBy` vs `findBy`?** Throws if missing / returns null (for asserting absence) / async, waits for it to appear.

6. **How did you test a live WebSocket app end-to-end?** Playwright `routeWebSocket` intercepts the Binance socket and the test pushes ticker messages in Binance's format. The app code runs unchanged, and the results are deterministic.
7. **What is a flaky test and how do you prevent one?** One that passes or fails randomly. Mock network and time, use auto-waiting assertions instead of sleeps, isolate state per test, and run the suite repeatedly to check.
8. **Is 100% coverage the goal?** No. Coverage shows what *ran*, not what was *checked*. Use it to find untested code, and aim for high coverage on critical logic.
9. **How do you test floating-point maths?** `toBeCloseTo` with a precision, known reference values, and invariants like put–call parity.
10. **What's a mock and when is it a bad idea?** A controlled fake for a dependency. It's bad when it mocks your own logic (then the test tests the mock), or when it drifts from the real API's behaviour.
11. **jsdom vs a real browser?** jsdom is a fast JS DOM with no layout, CSS or canvas. It's good for component logic. Use a real browser (Playwright) for layout, responsive behaviour and real user flows.